

# **Report – TWEB**

**2024/2025**

De Icco Riccardo, 1062555



**UNIVERSITÀ  
DI TORINO**

# 1. Introduction:

In this report, I present TWEB. The system is composed of a centralized **Main Gateway** (App.js) that orchestrates traffic between a **Frontend** client and two specialized backend services: a **Spring Boot Application** managing static relational data (PostgreSQL) and a **Node.js Microservice** managing dynamic data (MongoDB).

## 2. System Architecture

I designed a multi-tier solution to strictly separate concerns:

### 1. Data Layer:

- **PostgreSQL:** Stores structured, static data (Movies, Actors, Crew, Releases, etc.). I chose a relational database to ensure data integrity and efficient complex querying via SQL.
- **MongoDB:** Stores semi-structured, dynamic data (Reviews). This allows for high-speed ingestion and flexible schema management for the large volume of Rotten Tomatoes reviews.

### 2. Application Layer (Microservices):

- **Spring Boot Service (Port 8082):** Exposes REST APIs for the static data. I configured it using **Gradle** and JPA/Hibernate for ORM mapping.
- **Reviews Service (Port 3001):** A lightweight Node.js/Express server I developed specifically to query MongoDB and handle large datasets.
- **Main Gateway (Port 3000):** An Express.js server acting as a **Reverse Proxy**. I implemented this to unify the API surface for the frontend and to manage real-time communication via **Socket.io**.

### 3. Presentation Layer:

- A responsive web interface built with **HTML5**, **Bootstrap (Dark Mode)**, and **Handlebars.js** for client-side templating.

## 3. Implementation Details

### 3.1 Backend: Spring Boot (Static Data)

I structured the Java backend using the standard **Controller-Service-Repository** pattern.

- **Data Modeling:** I used JPA Entities with **Surrogate Keys** (Long id auto-increment) to handle relationships efficiently, decoupling the database internal IDs from the original CSV logic. I also implemented **DTOs (Data Transfer Objects)** to transfer data to the client, preventing the exposure of internal database structures.
- **Pagination:** Given the massive size of the datasets (e.g., 5.7M actors), I found it mandatory to implement **server-side pagination** (Pageable interface). All endpoints I created (e.g., /actors, /movies) support page and size parameters to prevent memory overflows and ensure responsiveness.
- **Complex Queries:** I implemented custom JPQL queries in the Repositories to filter movies by multiple criteria (rating, year range, duration) simultaneously.

### 3.2 Backend: Node.js & MongoDB (Reviews)

I designed the Reviews Microservice focusing on performance.

- **Data Ingestion:** I implemented a **Batch Insert Strategy** (chunk size: 5000) using csv-parser. This allows the server to ingest 1+ million reviews in seconds without blocking the event loop.
- **Smart Search:** I developed a search logic implementing a two-step strategy: it first attempts an **Exact Match** on the movie title (using regex anchors); if no results are found, it falls back to a **Partial Match**. This improves user experience by tolerating slight discrepancies in titles.
- **Path Management:** To ensure the code runs on any machine, I wrote a dynamic path resolution system that checks multiple locations (development vs. submission folder) to locate the CSV source files.

### 3.3 Main Gateway & Real-Time Chat

I implemented app.js as the single entry point for the application.

- **API Routing:** It forwards requests starting with /api to the Spring Boot server and requests for /reviews to the Mongo server. This design choice effectively solves **CORS issues** and simplifies the frontend logic.
- **Socket.io:** I implemented a real-time chat system where users can join "rooms" based on the Movie ID (socket.join(roomName)), allowing for topic-specific discussions isolated from other movies.

### 3.4 Frontend

I built the frontend as a Single Page Application (SPA)-like experience without heavy frameworks.

- **Technology Stack:** I relied on **Axios** for asynchronous HTTP requests and **Handlebars** for rendering data templates dynamically.
- **User Experience:** I designed a persistent Dark Mode using custom CSS variables. The "Movie Detail" page aggregates data from multiple sources (Java for details/cast, Mongo for reviews) and initializes the websocket connection for the chat only when needed.

## 4. Issues and Solutions (Data Ingestion)

One of the main challenges I faced was aligning the raw CSV data with the strict schema of the relational database.

- **Solution:** I developed Python scripts (postgres\_connection1.py) using **Pandas**. These scripts handle data cleaning, column renaming (e.g., studio -> studio\_name), and type conversion (dates to Integers/Years) before populating PostgreSQL via SQLAlchemy. This ensured that the Spring Boot application starts with a consistent dataset.

## 5. Conclusions

This project successfully meets all the requirements of the assignment. The separation of static and dynamic data proved to be an efficient design choice, allowing me to leverage the relational integrity of Postgres for core data and the speed of Mongo for reviews. The use of a Gateway pattern simplified the client-side architecture, and the inclusion of Socket.io added a modern, interactive layer to the platform.

## 6. Division of Work

**This project was carried out individually.** I was responsible for the entire software development lifecycle, specifically:

- **Architecture Design:** Defining the interaction between the Gateway, Microservices, and Databases.
- **Backend Development:** Implementing the Spring Boot application (Java) and the Node.js servers (Express).
- **Frontend Development:** Designing the UI/UX and writing the client-side logic (HTML/CSS/JS).

- **Data Engineering:** Writing the Python scripts for ETL (Extract, Transform, Load) processes to populate PostgreSQL and MongoDB.

## 7. Extra Information

To run and visualize the project, all servers must be started.

### Configuration:

1. **Databases:** Ensure PostgreSQL (port 5432) and MongoDB (port 27017) are active services.
2. **Spring Boot:** Update `backend_java/src/main/resources/application.properties` with local Postgres credentials.
3. **Data Population:** Run the Python scripts in the `/scripts` folder (via IDE run configuration) to populate Postgres. The Node.js server will automatically populate MongoDB on the first launch.

**Execution (via IntelliJ IDEA / WebStorm):** The project is designed to be executed directly from the IDE using the standard Run Configurations:

1. **Java Backend:** Open `SpringBootApplication.java` and click the **Run** button (Green Arrow) next to the `main` method. The server will start on Port 8082.
2. **Reviews Microservice:** Open `Server.js` (in the node backend folder), right-click on the file, and select **Run 'Server.js'**. The server will start on Port 3001.
3. **Main Gateway:** Open `app.js`, right-click on the file, and select **Run 'app.js'**. The server will start on Port 3000.
4. **Frontend:** Open `index.html` and click the IDE's browser icon (popup bar) or use the "Open in Browser" feature to launch the client.

## 8. Bibliography

- Spring Boot Documentation: <https://spring.io/projects/spring-boot>
- Mongoose Documentation: <https://mongoosejs.com/>
- Socket.io Documentation: <https://socket.io/docs/v4/>
- **Google Gemini.** Used for code debugging, architecture optimization, and drafting the technical report.